MEV & Constant Function Market Makers
Report 3

Milutin Popovic

1 Reinforcement Learning

This report is just going to be about the things I gathered relevant for solving my problem with Reinforcement learning, the focus is on the things I did not know until now. The first milestone would be to apply the AlphaZero algorithm on the problem. This means I have to know about Markov Decision Process and Monte Carlo Tree search. One of the questions faced is also how to encode the state, hence the need to fall back to Graph neural networks which utilize edge weights.

1.1 Markov Decision Process

In comparison with supervised learning, reinforcement learning is not fed directly with labeled data, instead it learns through an agent which takes actions and is rewarded based on the outcome of these actions. Hence, this method is better suited for graph based problems and learning games, where a decision needs to be made based on the environment. In these cases the combinatorial complexity of the game is too big, such that supervised learning would lack labeled data. For instance the Shannon number is 10^{120}, which is the lower bound of all possible positions in chess. In this chapter we will go through the foundation of reinforcement learning based no [1, Chapter 17].

The general learning scenario is based on the agent collecting information through actions, interacting with the environment. In response to an action the agent will enter a new state and will be given a reward depending on the general goal 1. For example the action that wins a decision wining game, the agent will get a reward of +1. The agents objective is to maximize the reward and by doing this determines the best curs of action at a given state, called the policy.

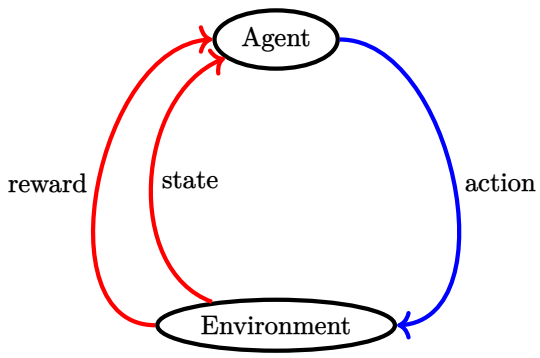

Figure 1: Reinforcement learning scenario

At each step the agent faces a decision to choose an action leading to a new state and the reward associated with it. Thereby the agent faces a dilemma in this choice based on the previous experience with the state and action, should the agent choose to export this information about the environment & states and optimize for the reward or should the agent explore other unknown states potentially leading to a more favorable outcome. In reinforcement learning this is called "exploration versus exploitation trade off.

Also, a key difference to traditional supervised learning, reinforcement learning does not sample features based on a fixed distribution. Essentially the training and testing phases are not separated, but rather combined in each step.

The two main learning settings are distinguished, the setting where the full environmental model is known and unknown.

The mathematical model which describes the agent's iteration with the environment is the Markov decision process (MPD).

Definition 1.1.1 *Markov decision process*

The MDP is defined by 4 tuple (S, A, δ, R_a):

- *The set of all states S, with a start state $s_0 \in S$,*
- *The set of all actions A,*
- *The transition probability $\mathbb{P}[s'|s,a] = \int_{S'} P_a(s,s')ds'$*
- *The reward probability $\mathbb{P}[r'|s,a] = \int_{S'} R_a(s',s)\,ds'$*

The transition probability $\mathbb{P}[s'|s,a]$ describes the probability of the agent landing in state s', when he is initially at state s and takes an action a. Similarity the reward probability is explained in the same way. The structure of this state transition only depends on the last state and the action taken and not on the whole history of states and actions hence the Markov property and the naming of the model. Usually the state transitions and rewards are deterministic, in this case the transition and reward probability is 1 and the MDP is called **deterministic** and the reward is $r(s,a)$. In the discrete model the agent will preform actions in a set of decision epochs $t \in 0, \dots, T$

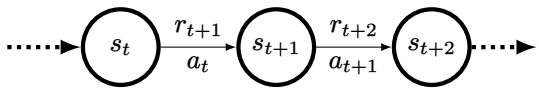

Figure 2: Deterministic discrete MDP [1]

2, this is the usual model adopted for a wide range of problems. The MDP is called **finite** when both S and A are finite sets. Furthermore, the MDP has a **finite time horizon** when T is finite.

The goal for the agent in an MPD is to find a good policy for the decision. The policy is a function, which given a state returns a probability distribution over the possible actions from the given state. The choice is then the action with the highest probability.

Definition 1.1.2 *Policy*

A policy is a mapping $\pi : S \to \Delta(A)$, where S is the state space and $\Delta(A)$ is the set of probability distributions over A.

We call the policy deterministic if:

$$\forall s \in S \;\; \exists! a \in A : \quad \pi(s)(a) = 1 \tag{1.1.1}$$

In this case we have $im(\pi) = A$, and we can say $\pi : S \to A$ with $\pi(s) = a$.

With the definition of a policy, the aim of the agent can be expressed as trying to find a policy that maximizes the total return. For a discrete policy the total return is

$$\sum_{t=0}^{T} r(s_t, \pi(s_t)) \qquad \text{finite horizon} \tag{1.1.2}$$

$$\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) \qquad \text{infinite horizon} \tag{1.1.3}$$

where $r(s_t, \pi(s_t))$ is the reward function for the next state and $\gamma \in [0, 1)$ a discount factor for future rewards.
To quantify what is the optimal policy, it is natural to define the value of the policy at a given state.

Definition 1.1.3 *Policy value*

The value $V_\pi(s)$ of the policy π at state $s \in S$ is defined as the expected value of the total reward over the policy distribution $\pi(s_t)$ of state $s_t \in S$ at epoch $t \leq T$. For the finite horizon it is

$$V_\pi(s) = \mathbb{E}_{a_t \sim \pi(s_t)} \left[\sum_{t=0}^{T} r(s_t, a_t) \middle| s_0 = s \right]. \tag{1.1.4}$$

For the infinite horizon for $\gamma \in [0, 1)$

$$V_\pi(s) = \mathbb{E}_{a_t \sim \pi(s_t)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \middle| s_0 = s \right]. \tag{1.1.5}$$

Thereby the optimal policy can be defined in the following way

Definition 1.1.4 *Optimal policy*

The policy π^* is called optimal if for any policy π it holds that

$$V_{\pi^*}(s) \geq V_\pi(s) \quad \forall s \in S. \tag{1.1.6}$$

Similarly to the policy value we can give the pair (s, a) associated to a policy π a value, the so called state-action value

Definition 1.1.5 *State-action value*

The state-action value function Q_π associated with a policy π is defined on all pairs state-action pairs $(s, a) \in S \times A$

$$\begin{aligned} Q_\pi(s, a) &= \mathbb{E}[r(s, a)] \mathbb{E}_{a_t \sim \pi(s_t)} \left[\sum_{t=1}^{\infty} \gamma^t r(s_t, a_t) \middle| s_0 = s,\ a_0 = a \right] \\ &= \mathbb{E} \left[r(s, a) + \gamma V_\pi(s_1) \middle| s_0 = s,\ a_0 = a \right]. \end{aligned} \tag{1.1.7}$$

Note that the expected value of the state-value function at (s, a) over the policy distribution at state s is the policy value at s, i.e.

$$\mathbb{E}_{a \sim \pi(s)} \left[Q_\pi(s, a) \right] = V_\pi(s). \tag{1.1.8}$$

With the help of these definitions it can be shown that for any MDP there is a deterministic optimal policy.

Theorem 1.1.1 *Policy Improvement Theorem*

For any two policies π' and π, it holds that

$$\begin{aligned} &\mathbb{E}_{a \sim \pi'(s)} \left[Q_\pi(s, a) \right] \geq \mathbb{E}_{a \sim \pi(s)} \left[Q_\pi(s, a) \right] \quad \forall s \in S \\ &\implies V_{\pi'}(s) \geq V_\pi(s) \quad \forall s \in S. \end{aligned} \tag{1.1.9}$$

Theorem 1.1.2 *Bellman's optimality condition*

A policy π is optimal if and only if for all state-action pairs $(s, a) \in SxA$ with $\pi(s)(a) > 0$ it holds that

$$a \in \arg\max_{a' \in A} Q_\pi(s, a'). \tag{1.1.10}$$

Theorem 1.1.3

Existence of optimal deterministic policy Any finite MDP has an optimal deterministic policy.

Consider an optimal deterministic policy π^* with its policy value and state-action value functions V^* and Q^*. The optimal policy for all state is

$$\pi^*(s) = \arg\max_{a \in A} Q^*(s, a). \tag{1.1.11}$$

Thus, the agent only needs to know the state-action value function Q^* to determine the optimal policy, and does not need to have information about the transition and reward probabilities. Using the definition of the state-value function for Q^*, $V^*(s) = Q^*(s, \pi^*(s))$ gives a system of equations called *Bellman equations*:

$$V^*(s) = \max_{a \in A} \left\{ \mathbb{E}[r(s,a)] + \gamma \sum_{s' \in S} \mathbb{P}[s'|s,a] V^*(s') \right\} \quad \forall s \in S. \tag{1.1.12}$$

Theorem 1.1.4 *Bellman Equations*

The policy value $V_\pi(s)$ of a policy π at state s for an MDP with infinite horizon is subjected to the following linear system of equations

$$V^*(s) = \mathbb{E}_{a_1 \sim \pi(s)}[r(s, a_1)] + \gamma \sum_{s' \in S} \mathbb{P}[s'|s, \pi(s)] V^*(s') \quad \forall s \in S. \tag{1.1.13}$$

We can rewrite the linear system

$$v = \gamma P v + r, \tag{1.1.14}$$

where

$$p_{s,s'} = \mathbb{P}[s'|s, \pi(s)] \quad \forall s, s' \in S \quad \text{is the transition probability matrix,} \tag{1.1.15}$$
$$v_s = V_\pi(s) \quad \forall s \in S \quad \text{is the value vector and} \tag{1.1.16}$$
$$r_s = \mathbb{E}[r(s, \pi(s)] \quad \forall s \in S \quad \text{is the reward vector.} \tag{1.1.17}$$

For an MDP with finite horizon with a known probability matrix P and a known reward expectation r, linear algebra gives a unique solution for the value vector by matrix inversion

$$v^* = (I - \gamma P)^{-1} R, \tag{1.1.18}$$

where I is the appropriate identity matrix.

Proofs for the theorems can be found in [1, Chapter 17].

1.2 Monte Carlo Tree Search

MCTS is a search algorithm for decision processes like MDP. The algorithm focuses on finding the most favorable course of action by uniformly expanding the search tree at each step and applying random actions/decisions until reaching a terminal state, this outcome is then communicated back to the start (backpropagation). In the last decade MCTS has been combined with learning based methods [2] to produce agents which can beat humans in Chess, GO and other games.

On a basic level the fundamental concept of the algorithm is that the true value of an action can be approximated by random simulation, which ultimately adjusts the policy efficiently to derive a best-first strategy [3]. One search iteration is done in four steps:

1. Selection Phase: Also called Tree-Traversal Phase, starts at the root node state and walks down until reaching a leaf node state with a selection policy. The leaf node state is one that is

not yet fully expanded, i.e. has unvisited children or is not a terminal state. The selection policy of the walk is done through the confidence bound ($UCB1$), where the idea is to weight out and balance the exploration vs exploitation. The next action is chosen such that the next node has the biggest $UCB1$ value. The basic $UCB1$ formula is

$$UCB1 = \frac{w_i}{n_i} + C\sqrt{\frac{\ln N_i}{n_i}}, \tag{1.2.1}$$

where w_i is the number of wins of the node, n_i the number of visits of the node, N_i the number of visits of the parent node and $C \in R$ is a constant, usually chosen to be $C = \sqrt{2}$

2. Node Expansion Phase: An extra node is added into the tree according to the available actions, if the node state is not a terminal one.

3. Simulation Phase: Also called Roll-out state. A simulation/roll-out is from the current node selecting actions randomly until a terminal node state.

4. Backpropagation Phase: The result of the simulation is used to update the information in the nodes traversed, such as the outcome/reward of the terminal state at the end of the simulation and also the visit count of the nodes.

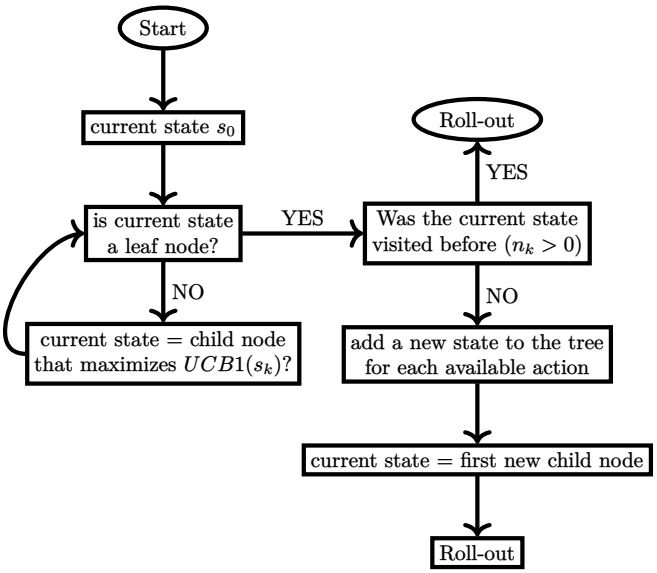

Figure 3: MCTS algorithmic diagram

The general algorithmic structure is described in [3], denoted in 1. For clarity purposes the diagram of the algorithm is in 3. The search starts with an input state s_0, and selects the next

Algorithm 1 MCTS Algorithm

Input: state s_0
  while within computational budget **do**
    $s_k \leftarrow$ SelectionPolicy(s_0)
    $w \leftarrow$ RollOut(s_k)
    BackPropagate(s_k, w)
  end while
Output: a (BestChild(s_0))

set of states based on the selection policy, which is are the states with the biggest $UCB1$ value. The state s_k is reached after the selection phase and the expansion, upon which a simulation is run giving the terminal result/outcome w. The outcome is then communicated back from s_k back

to the root node in the path taken to reach s_k from the root node. At the end the algorithm returns then the action a leading to the best child of s_0. The number of search iterations is usually indicated by a specific search time or number of search steps given and is denoted by the while statement.

1.3 Graph Neural Networks

This section is an introductory the Graph Neural Networks (GNNs) [4]. GNNs are Neural Networks (NNs) which operate over graph structured data. Let further denote a graph as a tuple (E, V), where E are edges and V are the nodes of the graph. Also let A be the adjacency matrix of the graph, where $a_{ij} = 1$ if $(i, j) \in E$ and $a_{ij} = 0$ otherwise. Finally, denote X to be the node features.

There are multiple types of GNNs some more complex than others [4], but the core principles are taken from convolutional NNs. An image can be seen as a grid graph, where each node corresponds to a pixel and is connected to its neighboring pixels. Simple convolution in images aggregates pixel by applying a simple operation (kernel) on its neighborhood, updating the pixels value. Similarly, in a graph the convolutional GNN will take the information of the neighboring node features (and edge features) to update the underlying node by applying some function on them 4.

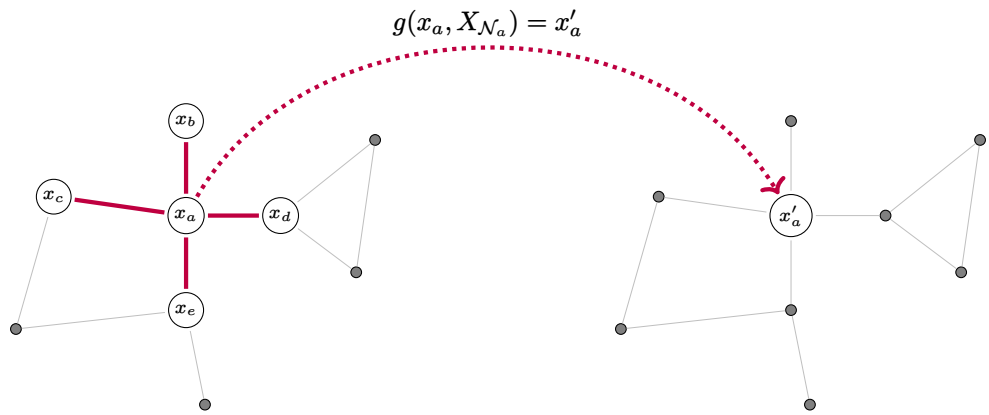

Figure 4: Convolutional GNNs: simple example

For a graph we can define a node's neighborhood, for a node $i \in V$ a nodes 1-hop neighborhood is

$$\mathcal{N}_i := \{j : (i, j) \in E \ \wedge \ (j, i) \in E\}. \tag{1.3.1}$$

The features of the 1-hop neighborhood $\mathcal{N}_i$ of the node are

$$X_{\mathcal{N}_i} = \{\{x_j : j \in \mathcal{N}_i\}\}. \tag{1.3.2}$$

Then the convolutional function g operates on the features of the node itself and on the features of its 1-hop neighborhood $g(x_i, X_{\mathcal{N}_i})$. Suitable functions that preserve the invariance or equivariance of the graph. When applying a permutation matrix P we need to permute the rows and the columns of the adjacency matrix A, which results to PAP^T, leading to functions satisfying

$$f(PX, PAP^T) = f(X, A) \quad \text{Invariance} \tag{1.3.3}$$

$$f(PX, PAP^T) = Pf(X, A) \quad \text{Equivariance} \tag{1.3.4}$$

By applying the local function g on all nodes and their neighborhoods we get

$$f(X, A) = \begin{bmatrix} g(x_1, X_{\mathcal{N}_1}) \\ \vdots \\ g(x_n, X_{\mathcal{N}_n}) \end{bmatrix}, \tag{1.3.5}$$

thus g needs to be permutation invariant, to not depend on the order in $X_{\mathcal{N}_i}$ and thereby f will be equivariant. Conventionally the local function g is referred to as "diffusion", "propagation" or "message passing" and the global function f as a "GNN Layer" [4, Chapter 5]. The majority of the GNN Layer types can be differentiated by three types, *Convolutional*, *Attentional* and *Message-passing* GNN. They are all different in the sense how g incorporates the neighboring features, where g needs to be permutation invariant.

The convolutional type [4, Chapter 5] as described above for clarification, the features of the neighbors are aggregated additionally with fixed weights c_{ij} after applying some function ψ on them

$$x_i' = g\left(x_i, \bigoplus_{j\in\mathcal{N}_i} c_{ij}\psi(x_j)\right). \tag{1.3.6}$$

In attention type [5] features of neighbors are applied with implicit weights, via attention. The attention coefficient for each pair is computed via a function $a_{ij} = a(x_i, x_j)$, usually a softmax function resulting in

$$x_i' = g\left(x_i, \bigoplus_{j\in\mathcal{N}_i} a(x_i, x_j)\psi(x_j)\right). \tag{1.3.7}$$

The message passing type [4, Chapter 5] computes arbitrary vector "messages", these are then sent through the edges back to the original node which gives

$$x_i' = g\left(x_i, \bigoplus_{j\in\mathcal{N}_i} \psi(x_i, x_j)\right). \tag{1.3.8}$$

In practice the functions g and ψ are learnable and are represented by some appropriate NN structure and the $\bigoplus$ operator is a nonparametric operation e.g. sum, mean, maximum or a recurrent neural network [4, Chapter 5].

Dealing with real world networks it is piratical to incorporate edge attributes in the GNN mechanism. For the attention type this is done with success in the original paper GAT [5] and improved on in [6] by allowing signed weights through the attention coefficient. The computation of the attention coefficient is the following

$$a_{ij}^k = \operatorname{sign}(e_{ij}) \cdot \operatorname{softmax}_j\left(\operatorname{abs}(e_{ij}^k)\right), \tag{1.3.9}$$

where

$$e_{ij}^k = \mathrm{MLP}^k\left(h_i^k \| h_j^k \| w_{ij}\right). \tag{1.3.10}$$

Here w_ij is the weight of the edge, MLP^k is a k-layer multilayer perceptron, where the last weight can produce negative values, h_i^k and h_j^k is the node feature embeddings after the k-th GNN layer. The input features are in the 0-th layer $h_i^0 = x_i$.

1.4 AlphaZero

The section is a brief overview of how AlphaZero [2] works. The special thing about AlphaZero is that it uses purely reinforcement learning to optimize the policy and there is are is no outside data fed in. There are two separate phases for making the model work with purely reinforcement learning, in the one phase the model plays with itself and gathers information about the environment and gathers the data. In the second phase the information of the self play part is used to optimize the model and then AlphaZero plays with the optimized model against itself again, forming a loop 5. The optimization is done through the parameters θ of a deep neural

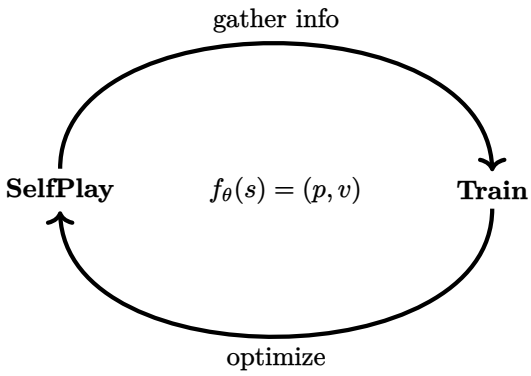

Figure 5: AlphaZero cycle

network $f_\theta(s) = (p, v)$, which takes a state as input and outputs the transition probability vector $p_a = \mathbb{P}[a|s]$ for each action and a scalar value v, which estimates the outcome of position s, $v \simeq \mathbb{E}[r|s]$. In the self-play phase the model uses the MCTS algorithm to traverse down the MDP tree from the root node s_0. The key difference in the walk to a leaf node is that alpha zero incorporate the output of $f_\theta(s)$ in UCB formula, which can be expressed as follows

$$UCB = Q(s, a) + U(s, a), \tag{1.4.1}$$

where $Q(s, a)$ is the state-action value, and $U(s, a) \propto \frac{P(s,a)}{1+N(s,a)}$ a value dependent on the node visit count $N(s, a)$ and probability $P(s, a)$ given by the deep neural network $f_\theta(s)$. Furthermore, AlphaZero does not utilize the third phase of the MCTS algorithm which randomly applies actions until reaching a terminal state. Instead, it just proceeds with the fourth phase of MCTS, of evaluating the current state and backpropagating to the root state. The architecture of f_θ is highly dependent on game/problem learned. Generally the first thing is state encoding, that is encoding the state in a usable way in AlphaGo Zero[2] the state is encoded in a $19x19x17$ array, instead of a simple $19x19$ board. The encoded state is passed residual neural network where the output is split into a value head neural network with zero-centered activation function to produce v and a policy head neural network with softmax activation function to output p. [TODO: mb picture here for the architecture]

2 Application to Problem

The problem finding a path starting and ending at the same node with no repeated edges which minimizes the weights is an NP-hard because it is a case of a traveling salesman problem (TSV). As a first exploration step AlphaZero is applied on the problem. The underlying MDP is a deterministic one, since taking an action corresponds using an edge to get to the next node. The MDP has also with finite horizon, dependent on the graph size. The agent starts at a node v_0 corresponding, follows a policy to take actions/edges leading to the next node v_1 until forming a circuit $\{v_0, v_1, \ldots, v_0\}$. The valid actions are edges from the current node v_i that have not yet been traversed. The state could be the path the agent is currently taking, i.e. a sequence of nodes $s = \{v_0, v_1, \ldots\}$. Upon arrival of the terminal state, which is either a circuit or a dead end the reward R must be assigned correspondingly. It makes since to keep a track of the agents "score", such that the model will try to find the path with the highest score. The score should be higher based on the sum of the weights and the number of actions taken. . The agent should be punished with a negative reward upon ending up at a terminal state which is not the desired circuit, i.e. dead end. So the total reward will be combined from a global reward (winning or losing) and a local reward (weight minimization/score)

Steps have been taken in implementing the problem in python and applying the MCTS algorithm on the problem. The current difficulties aeries when trying to implement AlphaZero because of the state encoding. The state encoding in graph based problems needs to be done through GNNs, but the output also needs to consider valid moves which are the edges not traversed. Hence, it

might make since to somehow encode the valid moves in the input through dynamic edges, i.e. removing edges. But will the removal of the edges even make since from a reinforcement learning perspective ? How should the input layer look like ?

References

[1] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014. Chap. 17.

[2] David Silver et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. 2017. arXiv: 1712.01815 [cs.AI]. URL: https://arxiv.org/abs/1712.01815.

[3] Cameron B. Browne et al. "A Survey of Monte Carlo Tree Search Methods". In: *IEEE Transactions on Computational Intelligence and AI in Games* 4.1 (2012), pp. 1–43. DOI: 10.1109/TCIAIG.2012.2186810.

[4] Michael M. Bronstein et al. "Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges". In: *CoRR* abs/2104.13478 (2021). arXiv: 2104.13478. URL: https://arxiv.org/abs/2104.13478.

[5] Petar Veličković et al. *Graph Attention Networks*. 2018. arXiv: 1710.10903 [stat.ML]. URL: https://arxiv.org/abs/1710.10903.

[6] Marco Grassia and Giuseppe Mangioni. "wsGAT: Weighted and Signed Graph Attention Networks for Link Prediction". In: *CoRR* abs/2109.11519 (2021). arXiv: 2109.11519. URL: https://arxiv.org/abs/2109.11519.

[7] Oleg Arenz. "Monte Carlo Chess". en. bachelor thesis. Darmstadt: Technische Universität Darmstadt, Nov. 2022, 35 Seiten. DOI: https://doi.org/10.26083/tuprints-00022930. URL: http://tuprints.ulb.tu-darmstadt.de/22930/.

[8] Raghuram Ramanujan, Ashish Sabharwal, and Bart Selman. "Understanding Sampling Style Adversarial Search Methods". In: *CoRR* abs/1203.4011 (2012). arXiv: 1203.4011. URL: http://arxiv.org/abs/1203.4011.

[9] Wouter Kool, Herke van Hoof, and Max Welling. *Attention, Learn to Solve Routing Problems!* 2019. arXiv: 1803.08475 [stat.ML]. URL: https://arxiv.org/abs/1803.08475.

[10] Qi Wang and Yongsheng Hao. "Routing optimization with Monte Carlo Tree Search-based multi-agent reinforcement learning". In: *Applied Intelligence* 53 (Aug. 2023), pp. 1–16. DOI: 10.1007/s10489-023-04881-1.